

RAG vs. LLM for ABINIT user support

A voting study on 100 questions from the ABINIT forum

Aldo Romero/Francesco Ricci/Gian Marcon Rignanese

Human up/down votes · full answer review · LLM-as-judge scoring

What this study is

- **ABINIT users were shown two answers per forum question and asked to up/down vote the one(s) they preferred.**
 - Answer A = Retrieval-Augmented Generation (RAG) grounded on an ABINIT knowledge base.
 - Answer B = a plain LLM answer (Gemini Flash) with no retrieval.
- **Voters were told to up-vote correct/useful answers and down-vote wrong, misleading or incomplete ones.**
- **Dataset: 100 questions randomly sampled from the old ABINIT forum; 76 questions received votes.**
- **This deck cross-references votes with the full answers, the forum-derived reference answer, and an automatic LLM-as-judge score.**

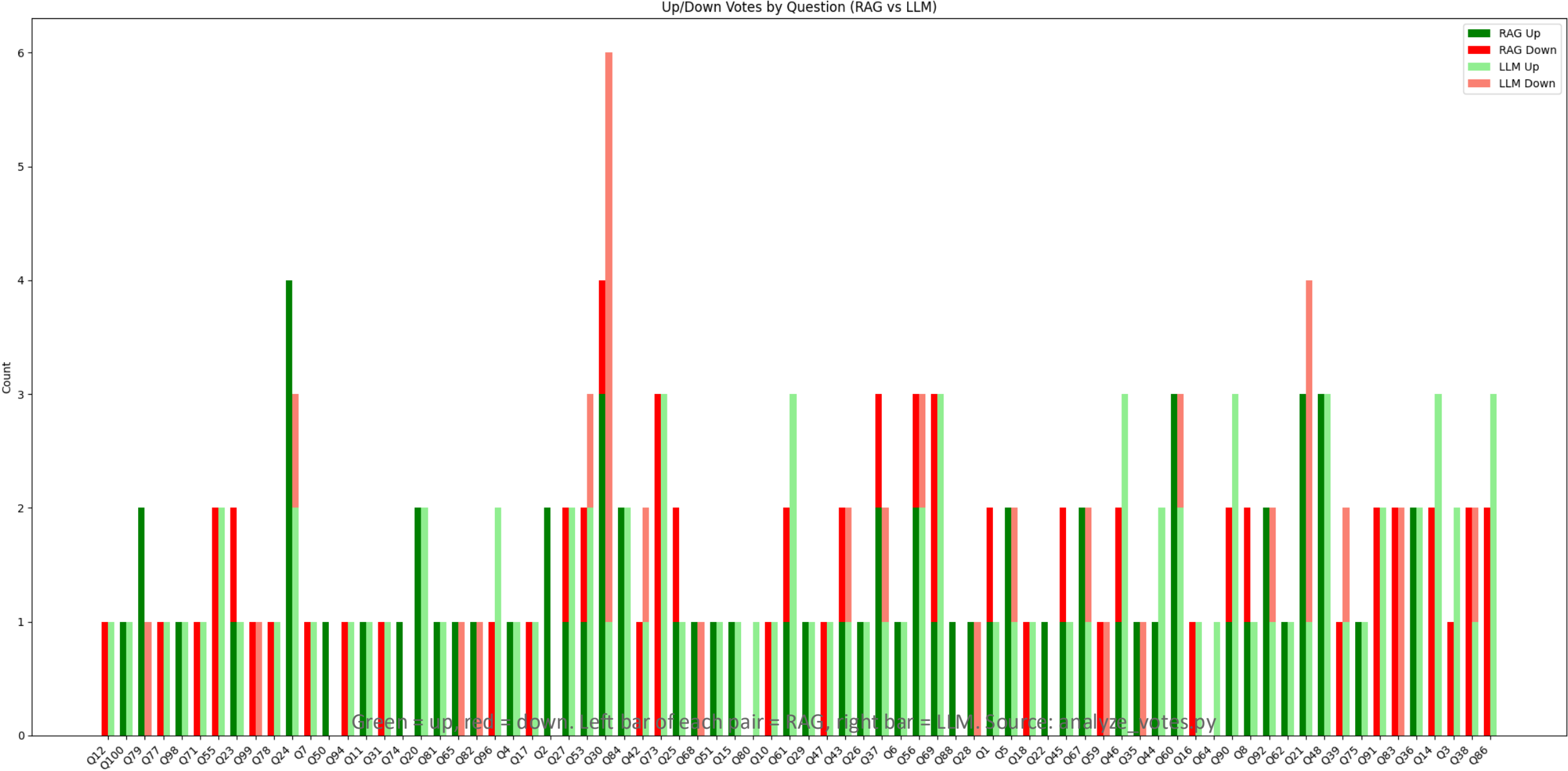
How questions were categorised

- Per question we compute a net score for each system: $\text{net} = (\text{up votes}) - (\text{down votes})$.
- Both down: $\text{RAG net} < 0$ AND $\text{LLM net} < 0 \rightarrow$ neither answer satisfied users (3 questions).
- LLM superior: $\text{LLM net} > 0$ AND $\text{RAG net} < 0 \rightarrow$ users preferred the plain LLM (20 questions).
- RAG superior: $\text{RAG net} > 0$ AND $\text{LLM net} < 0 \rightarrow$ users preferred the grounded RAG (8 questions).
- Remaining voted questions are ties / mixed and are not detailed individually (45).

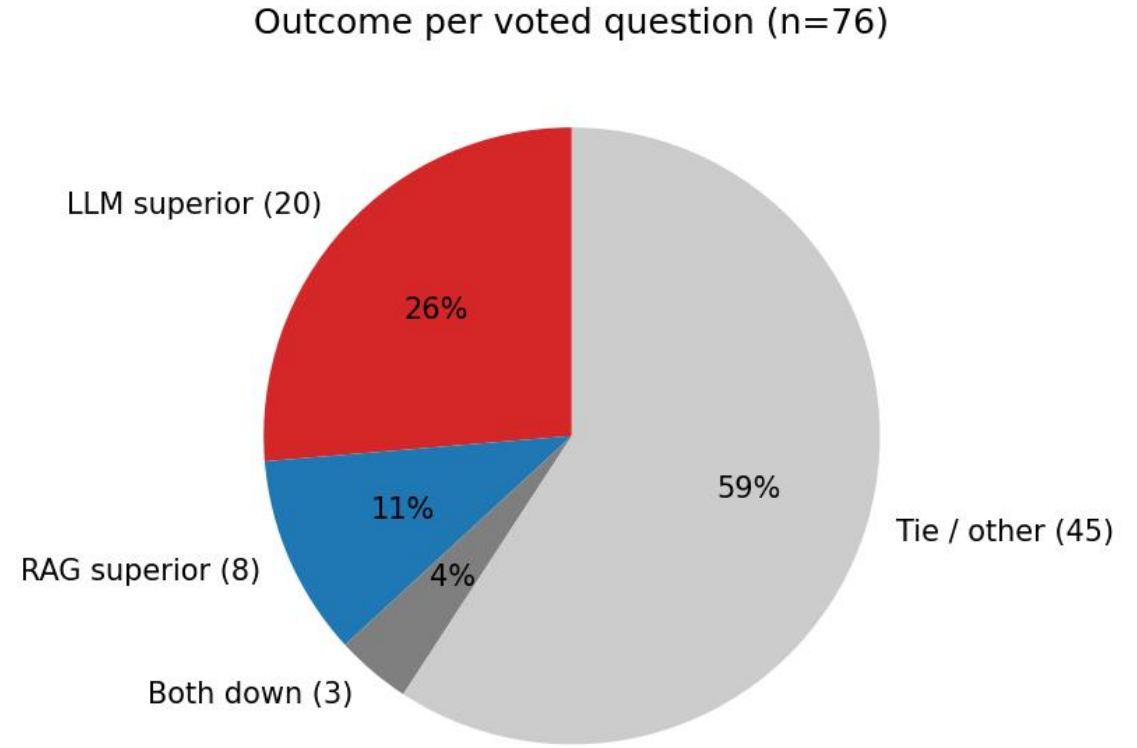
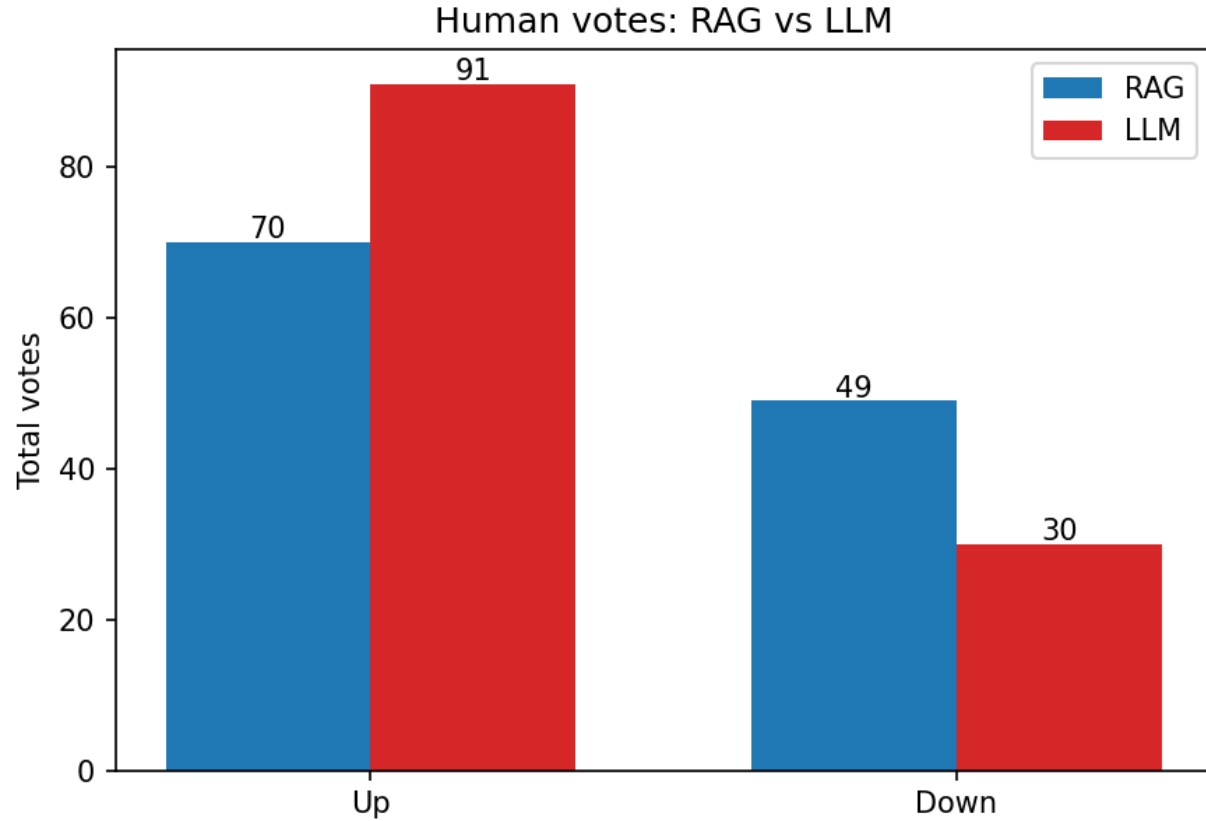
Aggregate results

What the votes and the automatic judge say across all 100 questions

Per-question votes (RAG vs LLM)

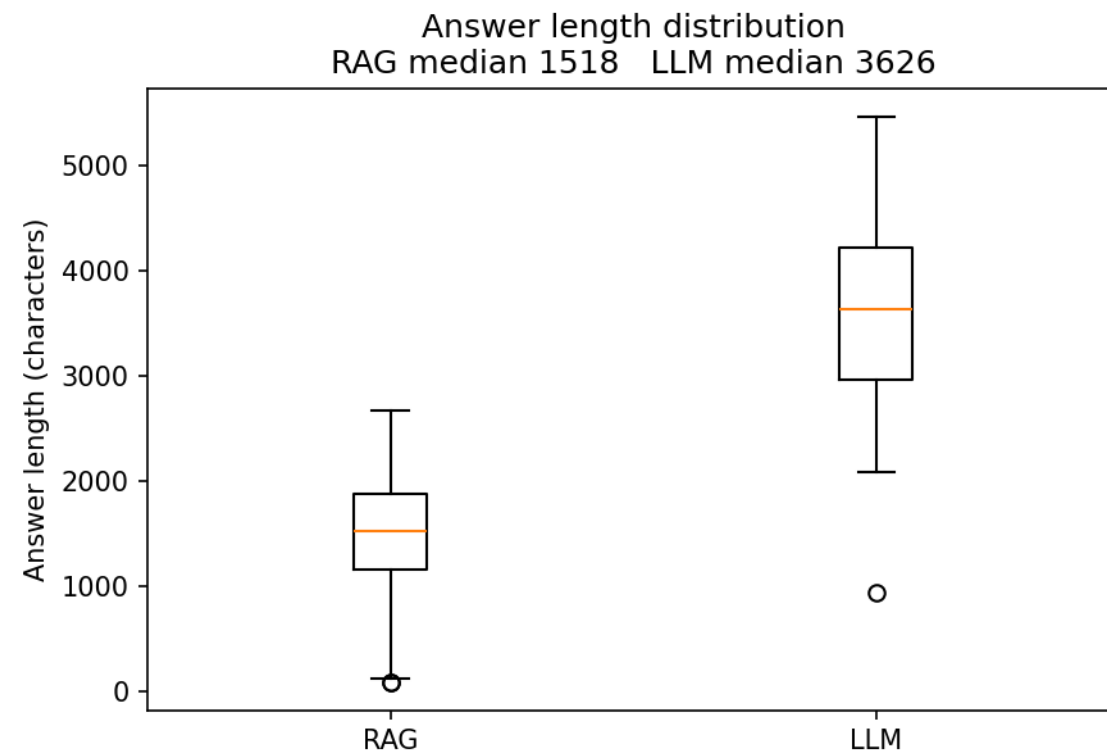
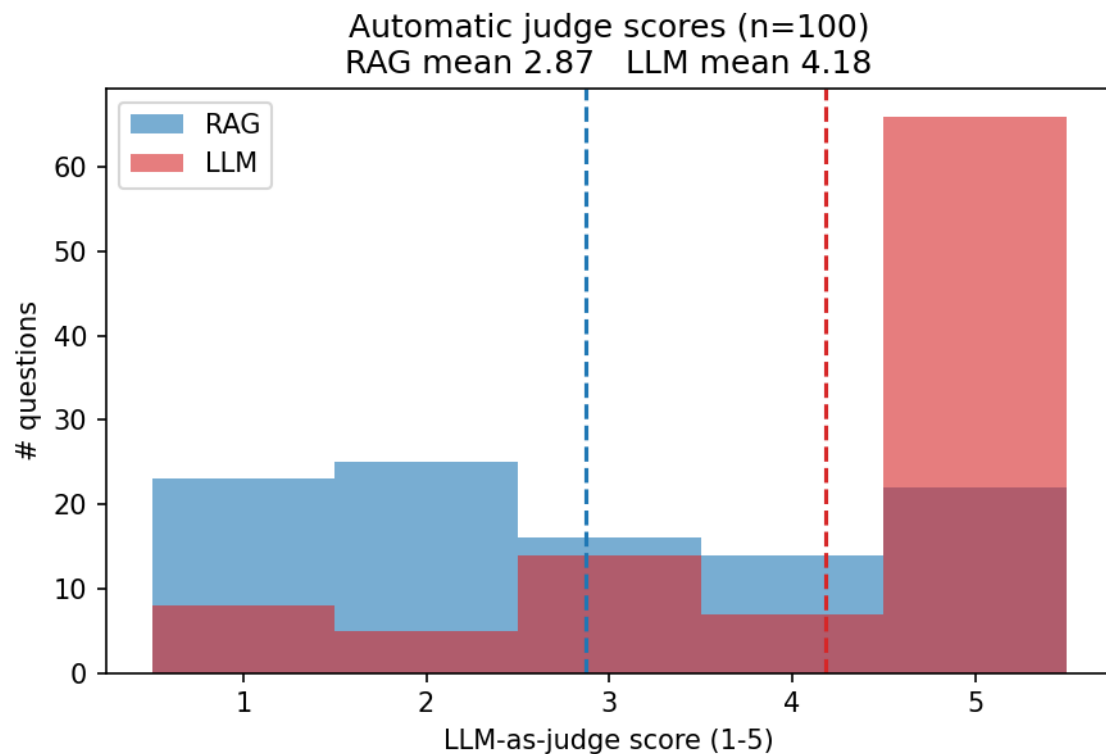


Vote tally and outcome split



LLM collected more up-votes and far fewer down-votes; it wins the majority of decided questions.

Automatic LLM-as-judge cross-check



Independent judge agrees with voters: LLM mean 4.18 vs RAG 2.87 / 5. RAG answers are also markedly shorter.

Headline findings

- Users up-voted the LLM more often (91 up / 30 down) than RAG (70 up / 49 down).
- The plain LLM was preferred on 20 voted questions; RAG was preferred on 8.
- Only 3 questions were down-voted on both systems — genuinely hard / unanswered cases.
- An independent LLM-as-judge agrees: mean score LLM 4.18 vs RAG 2.87 out of 5.
- 3 of the RAG answers were literally "I couldn't find any information..." — retrieval gaps drive several losses.
- Caveat: small and uneven vote counts per question; treat as qualitative signal, not statistics.

Down-voted on BOTH systems

3 questions where neither RAG nor LLM satisfied users — see both messages

Q83 — DOWN-VOTED ON BOTH

RAG ▲0 ▼2 | LLM ▲0 ▼2 Judge: RAG 3/5 LLM 3/5

QUESTION

Why does my processor distribution with npfft=11 fail with the error "ngfft is bigger than allowed value = 2" while npfft=10 works?

REFERENCE (forum-derived gold answer)

Because ABINIT's Goedecker FFT implementation requires npfft to be a multiple of 2 and 7; 10 satisfies this but 11 does not.

Resolution: Use an npfft that is a multiple of 2 and 7 (e.g., 10, 14, 20, etc.) and keep the npband×npfft product modest (≈ 300) to avoid the ngfft error.

Q83 — answers

RAG (judge 3/5)

The error you are encountering when setting ``npfft=11`` stems from the specific requirements for FFT parallelization in ABINIT. When using MPI-FFT (activated when ``npfft > 1`` under ``paral_kgb=1``), the dimensions of your FFT mesh (``ngfft``) must be compatible with the number of processors assigned to the FFT level (``npfft``).

In many parallel FFT implementations, the last two dimensions of the FFT mesh are required to be multiples of ``npfft``. When you change ``npfft``, the code automatically recalculates or adjusts the ``ngfft`` values to ensure the calculation remains valid—for instance, maintaining a ``boxcut`` value of at least 2.0 to prevent FFT filter errors.

The failure occurs because ``npfft=11`` results in an ``ngfft`` configuration that is either incompatible with the constraints of the FFT library or violates the code's internal parallelization logic. Conversely, ``npfft=10`` produces a grid configuration that satisfies these divisibility and validity requirements.

To resolve this, consider the following recommendations:

* **Use ``autoparal``:** The most reliable approach is to use the ``autoparal`` input keyword. This allows the code to automatically determine an optimal and valid value for ``npfft`` along with the corresponding ``ngfft`` dimensions.

LLM (judge 3/5)

Hello,

Thank you for reaching out with your question regarding ABINIT parallelization.

The error you are encountering is related to how ABINIT distributes the Fast Fourier Transform (FFT) grid across processors. Here is a clear and thorough explanation of why ``npfft=10`` works while ``npfft=11`` fails.

The Core Reason: Divisibility and FFT Grid Dimensions

In ABINIT, the FFT grid is defined by three dimensions, represented by the variable ``ngfft`` (e.g., $N_x \times N_y \times N_z$). To parallelize the FFT calculation, ABINIT distributes these grid planes (or columns) among the processors allocated for FFTs (``npfft``).

For this distribution to be efficient and mathematically valid:

- Divisibility Constraint:** The number of processors dedicated to the FFT (``npfft``) **must be a divisor** of the FFT grid dimensions (specifically the Z-dimension, N_z , for slab decomposition, or a combination of dimensions for pencil decomposition).
- FFT Grid Optimization:** To ensure fast Fourier transforms, ABINIT automatically chooses ``ngfft`` dimensions that are highly composite numbers—meaning they are products of small prime factors (typically 2, 3, 5, 7).

Q83 — automatic judge feedback

RAG · score 3/5

Feedback: The response fails to identify the specific technical constraint causing the error. The reference answer correctly points out that the Goedecker FFT routine in ABINIT has specific requirements regarding the factors of `npfft` (multiples of 2 and 7), which explains why 10 (2x5) works and 11 (a prime number) fails. The provided response offers generic advice about FFT parallelization and divisibility without mentioning the specific restriction of the Goedecker routine, which is the root cause of the error message provided by the user. Therefore, the response is only somewhat correct as it provides general troubleshooting steps but misses the factual explanation required to resolve the specific error.

LLM · score 3/5

Feedback: The response provides a logical explanation for why `npfft=11` might fail based on general principles of FFT parallelization (divisibility and prime factors). However, it is factually inaccurate regarding the specific constraints of the ABINIT software mentioned in the reference answer. The reference answer clarifies that the specific Goedecker FFT routine used by ABINIT has a hard constraint requiring grid sizes to be multiples of 2 and 7, whereas the response incorrectly suggests that ABINIT optimizes for factors of 2, 3, and 5. Because the response misses the specific technical constraint of the software in question, it is not completely accurate.

Q59 — DOWN-VOTED ON BOTH

RAG ▲0 ▼1 | LLM ▲0 ▼1 Judge: RAG 5/5 LLM 5/5

QUESTION

What is the constant that appears in the normalization of phonon eigendisplacements ($U(m)*M*U(n) = \delta(m,n)*0.0005485$) and what do the ';' and '-' symbols next to mode numbers mean?

REFERENCE (forum-derived gold answer)

The constant comes from ABINIT using electronic units; the mass is expressed in electron masses, so the normalization factor is $1/m_e \approx 0.0005485$. The ';' and '-' are just different print markers used by ABINIT for mode listings.

Resolution: The constant 0.0005485 is the inverse of the electron mass in atomic units, arising from ABINIT's use of electronic units. The ';' and '-' symbols are merely print markers.

Category: phonon eigendisplacement normalization

Q59 — answers (1/2)

RAG (judge 5/5)

The constant **0.0005485** that appears in the normalization of phonon eigendisplacements is the reciprocal of the conversion factor used to translate atomic mass units (amu) into atomic units (electron masses). Specifically, since one amu is approximately 1822.88851 electron masses, the value 0.0005485 is derived from $1 / 1822.88851 \approx 5.485 \times 10^{-4}$. This constant is necessary to maintain consistency in calculations where the mass matrix is defined in amu, but internal computations are performed using electron masses.

Regarding the symbols `;` and `-` appearing next to mode numbers, there is no explicit definition for these as specific markers for mode numbering. Instead, these characters typically function as standard formatting or structural elements within the software's output:

* **The `-` symbol:** This is most commonly used as part of standard numerical representation, such as indicating negative values or negative exponents in scientific notation (e.g., $.25d-4$). In other contexts, such as input variables like

LLM (judge 5/5)

Hello! It is a pleasure to assist you with your ABINIT calculations.

Below is a detailed and clear explanation of the normalization constant and the symbols you encountered in the phonon output.

1. The Normalization Constant (0.0005485)

The constant **0.0005485** (often rounded to 0.0005485) is the **mass of an electron expressed in Unified Atomic Mass Units (amu)**.

Why does it appear?

ABINIT performs its internal calculations using **Hartree atomic units**, where:

- * The mass of an electron (m_e) is defined as 1.0 .
- * The unit of length is the Bohr (a_0).

Q59 — automatic judge feedback

RAG · score 5/5

Feedback: The response correctly identifies the constant 0.0005485 as the reciprocal of the electron mass in atomic units (or the conversion factor between amu and electron masses). While the response uses a slightly different value for the electron mass (1822.88851) compared to the reference (1836.152), this is a minor discrepancy in physical constants often used in different versions or approximations within software packages. The explanation regarding the symbols ';' and '-' as formatting markers is accurate and aligns with the reference answer. The response is well-structured, factual, and addresses both parts of the user's question effectively.

LLM · score 5/5

Feedback: The response provides a highly accurate and detailed explanation of the normalization constant, correctly identifying it as the conversion factor between atomic mass units (amu) and electron mass (m_e) in Hartree atomic units. It correctly explains the physical significance of the normalization relation. Regarding the symbols, the response correctly identifies the semicolon as a formatting delimiter. While it interprets the minus sign as an indicator of imaginary frequencies (which is correct in the context of ABINIT phonon output), it provides a more comprehensive and helpful explanation than the reference answer, which dismisses the minus sign as having no physical meaning. The response is technically sound, well-structured, and provides additional context that is beneficial for a user performing ABINIT calculations.

Q99 — DOWN-VOTED ON BOTH

RAG ▲0 ▼1 | LLM ▲0 ▼1 Judge: RAG 3/5 LLM 5/5

QUESTION

Can the list of q-vectors generated for response function calculations be reduced by removing random points (e.g., every other q-point) without significantly affecting the results, especially for metals with fine k-point grids?

REFERENCE (forum-derived gold answer)

Yes, you can reduce the q-point list by removing points, but the remaining q-grid must still be a subgrid of the k-grid and you should test convergence. Typical q-grids are $4 \times 4 \times 4$ or $8 \times 8 \times 8$, and the choice depends on the system.

Resolution: You may reduce the q-point list by discarding points, provided the remaining q-grid remains a subgrid of the k-grid and you verify convergence. Typical grids of $4 \times 4 \times 4$ or $8 \times 8 \times 8$ are usually adequate, but convergence must be checked for each system.

Category: q-point generation for response functions

Q99 — answers (1/3)

RAG (judge 3/5)

When performing response function calculations for metals, it is generally not recommended to reduce the list of q -vectors by removing random points, such as taking every other point.

The current documentation emphasizes the importance of maintaining grid consistency. Specifically, for metallic systems, it is advised that the k and $k+q$ grids remain identical, accounting for periodicity in reciprocal space. Because subsampling the q -vector list would likely break the necessary alignment between these grids, it is not considered a standard or validated procedure.

Furthermore, response function calculations for metals often rely on fine k -point grids—sometimes implemented via ``use_k_fine`` or by reading fine grid wavefunctions—to support the interpolation techniques required for electron-phonon coupling or temperature-dependent calculations. While the documentation notes that further testing is still required to determine if other q -points might be permissible, the current guidance is to prioritize grid consistency to ensure

LLM (judge 5/5)

Dear Colleague,

Thank you for reaching out with this excellent and highly practical question regarding Density Functional Perturbation Theory (DFPT) calculations in ABINIT.

To answer your question directly: ****No, you cannot arbitrarily or randomly remove q -vectors (such as taking every other point) from a systematically generated grid without severely affecting the accuracy of your physical results.**** This is especially true for metals.

Below is a detailed explanation of why a complete, regular grid of q -points is mathematically and physically necessary, followed by recommended, rigorous ways to reduce computational cost in ABINIT.

Q99 — automatic judge feedback

RAG · score 3/5

Feedback: The response is somewhat correct but misses the nuance provided in the reference answer. While the response correctly emphasizes the importance of grid consistency (ensuring k and $k+q$ grids align), it takes an overly restrictive stance by stating it is "not recommended" to reduce the q -vector list. The reference answer clarifies that reducing the q -grid is acceptable provided the resulting set remains a proper subgrid of the k -grid and that convergence tests are performed. The response fails to mention that a coarser q -grid is often sufficient and physically valid for many systems, instead focusing solely on the risks of breaking grid alignment.

LLM · score 5/5

Feedback: The response provides a highly accurate and detailed explanation of why arbitrarily removing q -points from a grid is problematic in the context of DFPT calculations. It correctly identifies the necessity of a regular grid for Fourier interpolation and the specific challenges posed by metals, such as Kohn anomalies and the requirements for electron-phonon coupling calculations. The response also offers excellent, practical alternatives for reducing computational cost, such as using coarser grids and leveraging parallelization. While the reference answer mentions that a subgrid of the k -grid is the standard approach, the evaluated response correctly emphasizes the mathematical necessity of the grid structure and provides a more comprehensive guide for the user. The information is technically sound and aligns perfectly with the requirements of the task.

LLM was preferred

20 questions where the plain LLM beat RAG

Q73 — LLM SUPERIOR

RAG ▲0 ▼3 | LLM ▲3 ▼0 Judge: RAG 2/5 LLM 5/5

QUESTION

How to read Dynamical matrix elements (GKK File) – what is the dimension of the numbers, how do they relate to matrix elements, and how are they organized with respect to k-points?

REFERENCE (forum-derived gold answer)

The GKK file lists the dynamical matrix elements as real and imaginary parts for each pair of bands (n, n'), in order of k-points. The file is binary and each block corresponds to a single perturbation (q-point, atom, direction).

Resolution: To interpret a GKK file, read it as a binary stream of complex numbers (real, imag). The first dimension is the band pair index (n, n'), the second dimension is the k-point index. Each block in the file corresponds to a single perturbation (q-point, atom, direction).

Q73 — automatic judge feedback

RAG · score 2/5

Feedback: The response fails to provide the specific technical details requested by the user regarding the organization and dimensions of the GKK file. While the reference answer clearly explains that the data is stored as a sequence of complex numbers (real and imaginary parts) organized by k-points and band pairs, the response incorrectly claims that the internal structure is not documented and suggests the user must consult the source code. Consequently, the response does not answer the core of the user's question, making it inaccurate and unhelpful compared to the reference.

LLM · score 5/5

Feedback: The response provides a comprehensive and accurate explanation of the GKK file structure, its physical meaning, and how to access the data. It correctly identifies that the file contains electron-phonon matrix elements, uses atomic units, and is stored in a binary format. The response also correctly describes the hierarchical organization of the data (q-points, modes, k-points, and bands) and provides practical advice on using tools like AbiPy to handle the data. The information aligns well with the reference answer, providing additional helpful context while maintaining technical accuracy.

Q86 — LLM SUPERIOR

RAG ▲0 ▼2 | LLM ▲3 ▼0 Judge: RAG 3/5 LLM 5/5

QUESTION

Why does the topmost band appear noisy when using SOC with HGH pseudopotential, and how can I get rid of it?

REFERENCE (forum-derived gold answer)

The noise comes from insufficient convergence of the high-energy states; increase the number of bands, use a buffer (nbdbuf), and tighten the wave-function convergence criteria (tolwfr, tolvrs).

Resolution: Increase the number of bands, use a buffer (nbdbuf), and tighten tolwfr and tolvrs to converge the high-energy states, which eliminates the noisy topmost band.

Q86 — automatic judge feedback

RAG · score 3/5

Feedback: The response correctly identifies that the issue might be related to numerical convergence, but it fails to provide the specific, actionable technical solution required to resolve the "noisy" topmost band issue in the context of SOC calculations. The reference answer explains that this is a known artifact caused by the default convergence criteria focusing on occupied states, and it provides specific parameters (nband, nbdbuf, tolwfr, tolvrs) to fix it. The evaluated response suggests increasing the energy cut-off, which is a general troubleshooting step but does not address the specific mechanism of why the topmost band appears noisy when using SOC. Therefore, the response is only somewhat accurate as it misses the core technical explanation and the standard solution for this specific problem.

LLM · score 5/5

Feedback: The response provides a comprehensive and technically sound explanation for the noise observed in the topmost bands when using SOC with HGH pseudopotentials in ABINIT. It correctly identifies the role of basis set truncation and the "hardness" of the pseudopotentials. The suggested solutions—increasing the number of bands (`nband`), increasing the kinetic energy cutoff (`ecut`), and considering alternative pseudopotentials—are all standard and effective practices in DFT calculations. While the reference answer specifically highlights the importance of `nbdbuf` and tightening convergence tolerances (`tolwfr`, `tolvrs`) to ensure the high-energy states are properly converged, the response covers the core physical reasons and practical workarounds accurately. The response is highly informative, well-structured, and aligns with the expert understanding of the issue.

Q14 — LLM SUPERIOR

RAG ▲0 ▼2 | LLM ▲3 ▼0 Judge: RAG 4/5 LLM 5/5

QUESTION

Why does the SCF cycle become erratic for wide decorated graphene ribbons, and how can I fix it?

REFERENCE (forum-derived gold answer)

Adjust the convergence parameters, especially diemac, to improve SCF convergence.

Resolution: Set diemac to a suitable value (e.g., 0.5–0.6) to stabilize the SCF cycles for wide decorated graphene ribbons.

Q14 — automatic judge feedback

RAG · score 4/5

Feedback: The response provides a good overview of general SCF convergence strategies in ABINIT, such as adjusting `nline` and band counts. However, it misses the specific, highly effective solution mentioned in the reference answer regarding the use of `diemac` (dielectric screening) for wide ribbons. While the response is technically sound regarding general convergence troubleshooting, it fails to address the specific physical cause (internal electric fields/charge density mixing instability) and the primary fix for this specific system type as identified by the expert. Therefore, it is mostly correct but lacks the key technical insight provided in the reference.

LLM · score 5/5

Feedback: The response provides a comprehensive, accurate, and highly technical explanation of why SCF cycles become erratic in wide, decorated graphene nanoribbons. It correctly identifies the physical causes (charge sloshing, localized states, and magnetic degeneracy) and provides specific, actionable ABINIT input variables to mitigate these issues. The response aligns perfectly with the reference answer's emphasis on tuning mixing parameters (like `diemac` and `diemix`) while expanding on the context with additional professional advice on smearing and spin initialization. The information is factually sound and directly addresses the user's query with high expertise.

Q55 — LLM SUPERIOR

RAG ▲0 ▼2 | LLM ▲2 ▼0 Judge: RAG 1/5 LLM 3/5

QUESTION

Can I move directory "16_hideleave" to earlier in the build chain?

REFERENCE (forum-derived gold answer)

No, instead replace the ABI_UNUSED macro in m_xmpi with a dummy if-statement.

Resolution: Replace ABI_UNUSED(var) in m_xmpi with a dummy if-statement (e.g., if (.False.) write(std_out, var)).

Category: macro usage / build configuration

Q55 — automatic judge feedback

RAG · score 1/5

Feedback: The response fails to provide any helpful information, stating that it could not find information about the directory in the provided sources. In contrast, the reference answer identifies the technical context (the `ABI_UNUSED` macro and circular dependencies) and provides a specific, actionable workaround. The response is completely unhelpful and fails to address the user's query.

LLM · score 3/5

Feedback: The response provides a general, high-level explanation of how the ABINIT build system works and correctly identifies that moving a directory is technically possible but highly discouraged due to dependency risks. However, it fails to address the specific technical context implied by the user's question, which relates to the ``ABI_UNUSED`` macro and the ``m_xmpi`` module—a common issue in ABINIT development. The reference answer provides a specific, actionable workaround for the underlying problem (suppressing unused variable warnings without breaking the build chain), whereas the response suggests a risky and complex directory restructuring. While the response is technically sound regarding general build system principles, it misses the specific expert-level solution required for this common ABINIT issue.

Q69 — LLM SUPERIOR

RAG ▲1 ▼2 | LLM ▲3 ▼0 Judge: RAG 2/5 LLM 5/5

QUESTION

Is there a way to start a SCF calculation from a provided charge density? If I use getden it reads the density only for nscf calculations.

REFERENCE (forum-derived gold answer)

Use irdden 1 to read a DEN file from a previous run.

Resolution: Set irdden = 1 and provide the DEN file from a previous run; this will start the SCF calculation with the supplied charge density.

Category: SCF initialization from existing charge density

Q69 — automatic judge feedback

RAG · score 2/5

Feedback: The response fails to address the user's question regarding Abinit, which is the context implied by the user's mention of ``getden``. Instead of providing the correct Abinit variable (``irdden``), the response incorrectly assumes the user might be using Quantum ESPRESSO and provides irrelevant information about that software. The reference answer correctly identifies that ``irdden 1`` is the standard way to initialize an SCF calculation from a density file in Abinit. By focusing on the wrong software and failing to provide the specific variable needed for the task, the response is not helpful or accurate for the user's specific query.

LLM · score 5/5

Feedback: The response is accurate and provides a comprehensive guide on how to initialize an SCF calculation from a provided charge density in ABINIT. It correctly identifies both the multi-dataset approach (``getden``) and the external file approach (``irden``), while also addressing the user's misconception regarding the utility of these variables. The technical considerations provided are also correct and helpful for a user attempting this task. The response is factual and aligns perfectly with the reference answer.

Q91 — LLM SUPERIOR

RAG ▲0 ▼2 | LLM ▲2 ▼0 Judge: RAG 3/5 LLM 5/5

QUESTION

Why does my ABINIT calculation for optimizing the interwall distance between (3,3) carbon nanotubes fail to converge for some distances and show a monotonically decreasing energy trend, and how can I fix it?

REFERENCE (forum-derived gold answer)

Use LDA pseudopotentials, increase the plane-wave cutoff, set `iprcel` to 45, and employ a denser k-point grid along the tube axis; with the correct PSP file the relaxation converges to the literature value.

Resolution: Use LDA pseudopotentials, raise the cutoff, set `iprcel` to 45, and use a denser k-point grid along the tube axis; with the correct PSP file the relaxation converges to the literature value.

Category: ABINIT convergence and pseudopotential selection for nanotube calculations

Q91 — automatic judge feedback

RAG · score 3/5

Feedback: The response provides a good general overview of why DFT calculations in ABINIT might fail to converge (e.g., basis set size, box size, optimization algorithms). However, it fails to address the specific physical and technical nuances relevant to the user's problem regarding (3,3) carbon nanotubes, which are highlighted in the reference answer. Specifically, the reference answer identifies critical issues such as the choice of exchange-correlation functional (GGA vs. LDA for carbon layer binding), specific parameter settings like ``iprcel``, and the necessity of sufficient k-point sampling along the tube axis. The response provides generic troubleshooting advice rather than the specific, expert-level technical corrections required to solve the convergence failure for this specific system.

LLM · score 5/5

Feedback: The response provides a highly accurate and technically sound explanation for the convergence issues encountered in ABINIT when simulating (3,3) carbon nanotubes. It correctly identifies the lack of dispersion corrections (vdW) as the primary reason for the lack of an energy minimum and correctly attributes the SCF convergence failures to the metallic nature of the system, suggesting appropriate remedies like electronic smearing and robust mixing algorithms (Pulay mixing). While the reference answer mentions specific issues from a particular case study (such as incorrect pseudopotential files and specific cutoff values), the response provides a comprehensive and generalized set of best practices that are physically correct and directly address the user's problem. The advice is actionable, follows standard computational chemistry practices for DFT, and is consistent with the physics described in the reference.

RAG was preferred

8 questions where grounded RAG beat the plain LLM

Q30 — RAG SUPERIOR

RAG ▲3 ▼1 | LLM ▲1 ▼5 Judge: RAG 5/5 LLM 1/5

QUESTION

How do I make the .ac9 file to work? what directory do I put it in? and how does ABINIT know where to look for it if I select a random name for it?

REFERENCE (forum-derived gold answer)

Create a .ac9 file with the required paths and run ./configure with the option --with-config-file="yourfile.ac9".

Resolution: Create a .ac9 file with the necessary library paths and invoke configure with the --with-config-file option pointing to that file.

Category: configuration

Q30 — answers (1/3)

RAG (judge 5/5)

In ABINIT, an `.ac9`` file is a configuration file used to centralize your build parameters, such as compiler options and library paths. To make it work, you generally create a file named ``hostname.ac9`` (where ``hostname`` is the name of your machine) and place it in a location where the build system can find it.

Where to place the file

The build system searches for your configuration file in specific directories based on a set order of precedence. If you place the file in one of these locations, the system will detect it automatically:

1. ****Your build directory:**** (e.g., ``~/abinit/tmp``)
2. ****The ``~abinit`` directory:**** Files placed here are preferred over those in the ``~/abinit/build`` directory.
3. ****``~/abinit/build``:** Placing your file here allows the ``configure`` script to automatically use it for new ABINIT installations.

Using a custom name or location

If you choose a random name for your file or want to store it in a non-standard location, the build system will not automatically detect it. In this case, you must explicitly point the ``configure`` script to your file during the build process. After initializing your build environment with ``./config/scripts/makemake``, you can run the ``configure`` script using the ``--with-config-file`` flag:

LLM (judge 1/5)

Hello! I would be delighted to help you with your ABINIT calculations.

To guide you accurately, it is important to note that ****`.ac9`` is not a standard file extension in ABINIT****. It is highly likely that this is a typo or a specific naming convention used in your research group.

Based on standard ABINIT workflows, you are most likely referring to one of three things:

1. ****A `.nc`` (NetCDF) file\*\*** (such as a wavefunction ``_WFK.nc`` or density ``_DEN.nc`` file used to restart or continue a calculation).
2. ****The `.files`` file**** (the master file that directs ABINIT to all other files).
3. ****A Pseudopotential file**** (which can sometimes have custom extensions).

Below is a thorough explanation of how ABINIT locates and uses these files, regardless of what you name them or where you put them.

Scenario 1: If it is a NetCDF (`.nc``) file (e.g., Wavefunctions or Density)

If you are trying to read an existing wavefunction or density file (often

Q30 — automatic judge feedback

RAG · score 5/5

Feedback: The response is accurate and provides a comprehensive explanation of how ABINIT handles `.ac9` configuration files. It correctly identifies that while there are default search locations for these files, the user can use a custom name or location by explicitly passing the `--with-config-file` flag to the `configure` script. This aligns perfectly with the reference answer's core instruction. The response is well-structured, factual, and directly addresses all parts of the user's query.

LLM · score 1/5

Feedback: The response is completely incorrect regarding the context of the `.ac9` file in ABINIT. The user is asking about the configuration file used during the compilation/installation phase of ABINIT (often named with an `.ac9` extension to denote ABINIT Configuration version 9), whereas the model incorrectly assumes the user is asking about runtime input files (like NetCDF files or pseudopotentials). Because the model fails to identify the correct purpose of the file and provides irrelevant instructions for runtime workflows instead of the installation process, it does not answer the user's question accurately.

Q21 — RAG SUPERIOR

RAG ▲3 ▼0 | LLM ▲1 ▼3 Judge: RAG 1/5 LLM 5/5

QUESTION

Why does ABINIT give a Fortran runtime error 'End of file' at fort.9 when running tutorial 1 on the H2 atom, and how can I fix it?

REFERENCE (forum-derived gold answer)

Specify the full absolute path to the pseudopotential file; tilde (~) expansion may not work in ABINIT's file opening.

Resolution: Use the full absolute path to the pseudopotential file in the ABINIT input; avoid relying on tilde expansion.

Category: pseudopotential_path

Q21 — automatic judge feedback

RAG · score 1/5

Feedback: The response fails to identify the specific, well-known cause of this error in the ABINIT H2 tutorial, which is the failure of the Fortran runtime to expand the tilde (~) character in file paths for pseudopotentials. Instead, the response provides generic troubleshooting advice (checking logs, simplifying execution, verifying setup) that does not address the root cause. The reference answer provides the precise technical explanation and the specific fix required, whereas the evaluated response offers only vague suggestions that are unlikely to resolve the issue.

LLM · score 5/5

Feedback: The response provides a comprehensive and accurate explanation of why the 'End of file' error occurs in ABINIT. While the reference answer specifically highlights the issue of tilde expansion in file paths, the response covers this (via the "Incorrect Path" point) and expands upon other common, practical causes such as Windows/Linux line ending incompatibilities and file corruption. The troubleshooting steps provided are highly relevant, technically sound, and follow best practices for debugging Fortran-based scientific software. The response is well-structured, easy to follow, and addresses the user's problem effectively.

Q79 — RAG SUPERIOR

RAG ▲2 ▼0 | LLM ▲0 ▼1 Judge: RAG 2/5 LLM 3/5

QUESTION

Anyone knows what the source of this error? I have tried to recompile but haven't managed to solve it.

REFERENCE (forum-derived gold answer)

Link ABINIT with MKL BLAS/LAPACK instead of the standard libraries.

Resolution: Compile ABINIT with gfortran and link it to the Intel MKL BLAS/LAPACK libraries. This resolves the unitarity error in the Wannier90 tests.

Category: ABINIT-Wannier90 integration / BLAS/LAPACK linking issue

Q79 — automatic judge feedback

RAG · score 2/5

Feedback: The user's prompt failed to include the specific error message or context, which is a common issue in technical support queries. While the reference answer provides a specific solution for a known error (likely implied by the context of the original source material), the model's response correctly identifies that it cannot provide a specific diagnosis without the necessary information. However, because the model failed to acknowledge the context of the user's query (which likely contained an image or attachment that the model could not see, or was part of a thread where the error was previously known), it missed the opportunity to be helpful. Given the rubric focuses on accuracy based on the reference, the model is technically accurate in its assessment of the provided input, but it fails to address the specific technical issue identified in the reference answer.

LLM · score 3/5

Feedback: The response is helpful and provides good general troubleshooting advice for ABINIT compilation issues. However, it fails to address the specific user request because the user did not provide the error message, and the model did not have enough information to diagnose the specific problem. While the model correctly identifies that it needs more information, it does not provide the specific technical insight found in the reference answer regarding the Wannier90 unitary matrix error. Therefore, it cannot be considered "completely correct" regarding the specific (though unstated) error the user is likely facing.

Q28 — RAG SUPERIOR

RAG ▲1 ▼0 | LLM ▲0 ▼1 Judge: RAG 5/5 LLM 5/5

QUESTION

Is there any variable in ABINIT that can output the partial DOS of each atom (including all orbitals)? If not, can I calculate it based on angular-momentum projected DOS through any code or software?

REFERENCE (forum-derived gold answer)

ABINIT does not provide a single variable for the total atom-resolved DOS; you must sum the l-projected PDOS manually or write a post-processing script.

Resolution: No, ABINIT does not directly output the total atom-resolved DOS; sum the l-projected PDOS manually or via a custom script.

Category: PDOS / atom-resolved DOS

RAG · score 5/5

Feedback: The response is accurate and directly addresses the user's question. It correctly identifies that ABINIT provides angular-momentum projected DOS (via `prtdos=3`) and confirms that there is no single variable to output the total atom-projected DOS automatically, requiring the user to sum the individual ℓ -channels. The response provides helpful guidance on how to perform this summation using external tools, which aligns with the practical requirements of the task. It is factually correct and consistent with the reference answer.

LLM · score 5/5

Feedback: The response is completely correct, accurate, and factual. It directly addresses the user's question by confirming that ABINIT does not have a single variable to output the total atom-resolved DOS, but that it can be achieved by summing the angular-momentum projected DOS (obtained via `prtdos = 3`). The response provides the correct input variables, explains the output format, and offers practical, accurate methods for post-processing the data, which aligns perfectly with the reference answer.

Q35 — RAG SUPERIOR

RAG ▲1 ▼0 | LLM ▲0 ▼1 Judge: RAG 5/5 LLM 5/5

QUESTION

Where are the utilities after installation? I cannot find the utilities after installing 7.4.3. Where are they? Also, is it serious that Abinit does not have any post-processing tool anymore to plot band structure?

REFERENCE (forum-derived gold answer)

The utilities are in scripts/post_processing/AbinitBandStructureMaker.py (renamed from AbinitBandMaker.py).

Resolution: The utilities are located in scripts/post_processing/AbinitBandStructureMaker.py.

Category: post-processing utilities location

Q35 — automatic judge feedback

RAG · score 5/5

Feedback: The response is accurate and provides helpful, relevant information regarding the location of utilities and post-processing tools for Abinit 7.4.3. It correctly identifies the ``scripts/post_processing`` directory mentioned in the reference answer and provides additional context regarding Abipy and other tools, which directly addresses the user's concern about the lack of plotting capabilities. The information is factual and aligns well with the reference answer.

LLM · score 5/5

Feedback: The response is completely correct, accurate, and factual. It directly addresses the user's question by identifying the location of the utilities in version 7.4.3 (specifically noting the distinction between compiled binaries and scripts) and provides a helpful, accurate explanation regarding the post-processing tools, including the specific script mentioned in the reference answer. The additional context provided about AbiPy and modern versions of ABINIT is also highly relevant and helpful.

Q65 — RAG SUPERIOR

RAG ▲1 ▼0 | LLM ▲0 ▼1 Judge: RAG 5/5 LLM 5/5

QUESTION

How can I obtain phonon eigenvectors in ABINIT (PHVEC file) and visualize them for each mode?

REFERENCE (forum-derived gold answer)

Set the variable eivec=2 to print the eigenvectors; you can then use tools like animemd to visualize the modes from the anadddb output.

Resolution: Use eivec=2 to print eigenvectors; then visualize them with animemd (or other compatible tools).

Category: phonon eigenvectors and visualization

Q65 — automatic judge feedback

RAG · score 5/5

Feedback: The response provides a comprehensive and technically accurate guide to obtaining and visualizing phonon eigenvectors in ABINIT. It correctly identifies the use of the ``ANADDB`` utility and the ``eivec`` variable for generating eigenvector data. Furthermore, it offers modern, practical solutions for visualization using the ``AbiPy`` ecosystem, which is the standard approach for current ABINIT users. While the reference answer suggests the older ``animemd`` tool, the response's inclusion of ``AbiPy`` and ``VESTA`` integration is highly relevant and accurate for contemporary workflows. The information is well-structured, factual, and directly addresses the user's request.

LLM · score 5/5

Feedback: The response provides a comprehensive and practical guide for obtaining and visualizing phonon eigenvectors in ABINIT. While the reference answer suggests using ``eivec = 2`` and the ``animemd`` tool, the response correctly identifies that the modern and most efficient workflow in ABINIT involves using ``anaddb`` with ``prtphof = 1`` to generate ``.axsf`` files, which are widely compatible with standard visualization software like XCrySDen and VESTA. The response is technically accurate, well-structured, and provides a more user-friendly approach than the reference answer, making it highly useful for a researcher.

Q68 — RAG SUPERIOR

RAG ▲1 ▼0 | LLM ▲0 ▼1 Judge: RAG 2/5 LLM 2/5

QUESTION

How to switch off the local-field effect in GW calculation?

REFERENCE (forum-derived gold answer)

There is no built-in switch in ABINIT; the usual workaround is to delete the off-diagonal $G \neq G'$ terms of the susceptibility (χ) after the calculation, e.g. with a small script.

Resolution: No built-in switch; delete off-diagonal $G \neq G'$ terms of the susceptibility file manually to suppress the local-field effect.

Category: usage

Q68 — automatic judge feedback

RAG · score 2/5

Feedback: The response is inaccurate because it suggests that the local-field effect can be switched off by simply setting an input variable (`bs_exchange_term = 0`) or using specific calculation modes. As noted in the reference answer, the local-field effect in GW calculations is fundamentally tied to the off-diagonal elements of the dielectric matrix. ABINIT does not provide a simple runtime flag to disable this; rather, it requires post-processing the dielectric matrix (setting off-diagonal elements to zero) and re-running the calculation. The response provides misleading information regarding the mechanism to disable these effects.

LLM · score 2/5

Feedback: The response is incorrect. It suggests that setting the `ecuteps` parameter to a very small value in ABINIT will effectively switch off local-field effects (LFEs) by restricting the dielectric matrix to the $\mathbf{G}=\mathbf{G}'=0$ component. However, as noted in the reference answer, ABINIT does not provide a simple runtime flag or input parameter like `ecuteps` to achieve this. The standard and correct procedure to remove LFEs in ABINIT is to post-process the susceptibility or dielectric matrix files to zero out the off-diagonal elements ($\mathbf{G} \neq \mathbf{G}'$) and then use the modified file for the subsequent GW calculation. The response provides a misleading technical solution that does not align with the software's actual capabilities.

Q82 — RAG SUPERIOR

RAG ▲1 ▼0 | LLM ▲0 ▼1 Judge: RAG 4/5 LLM 5/5

QUESTION

How can I obtain the total magnetization from an ABINIT spin-polarized calculation of 3d transition-metal dimers on surfaces?

REFERENCE (forum-derived gold answer)

Set a non-zero spinat for the magnetic atoms, use nsppol=2, choose a metallic occupation (occopt≥3 with tsmear), and ABINIT will print the total magnetization (mag* lines) in the log file. For PAW you may also set prtdensph=1 to see it more clearly.

Resolution: Use nsppol=2, set spinat for magnetic atoms, choose a metallic occupation (occopt≥3 with tsmear), and read the mag* lines in the log. For PAW add prtdensph=1 if needed.

Category: magnetization

Q82 — automatic judge feedback

RAG · score 4/5

Feedback: The response provides a good overview of the necessary ABINIT variables (`nsppol`, `nspden`, `spinat`) to set up a spin-polarized calculation. However, it is less accurate than the reference answer regarding how to obtain the total magnetization. Specifically, it suggests that post-processing with `cut3d` is a primary way to extract the total magnetization, whereas the reference answer correctly points out that the total magnetization is automatically computed and printed in the standard output file during the SCF cycle. Furthermore, the response misses the crucial requirement of using a metallic occupation scheme (`occopt >= 3`) and smearing (`tsmear`), which are essential for transition-metal dimers on surfaces to ensure the spin channels are treated correctly. While the response is helpful, it lacks the technical precision regarding the standard output and necessary occupation settings found in the reference.

LLM · score 5/5

Feedback: The response is highly accurate, well-structured, and provides clear, actionable instructions for obtaining both total and local magnetization in ABINIT. It correctly identifies the necessity of `nsppol = 2` and the use of `spinat` for spin-polarized calculations. The explanation for locating the total magnetization in the output file is correct and matches the standard ABINIT output format. Furthermore, the response provides helpful, advanced guidance on using PAW spheres and Hirshfeld analysis for site-projected magnetization, which adds significant value beyond the basic requirements. It is technically sound and directly addresses the user's query with professional clarity.

Takeaways

- On this forum sample the plain LLM (Gemini Flash) was preferred more often than the RAG pipeline.
- RAG's main failure mode is retrieval: when the KB lacks the answer it returns short or empty responses that users reject.
- RAG still wins where grounding matters — version-specific behaviour, file locations, exact variable names (see RAG-superior set).
- Next steps: improve the knowledge base coverage, fall back to the LLM when retrieval is weak, and collect more votes per question.